On-Prem Keycloak with Active Directory and OIDC Single Sign-On

Technical guide for client discovery and delivery

Author: Talha Bilal | Portfolio sample | Version 1.0

1. Executive summary

This document explains how a typical small-to-mid-size organization achieves:

- **On-prem Keycloak** as the identity and access management (IAM) hub
- **Active Directory (AD)** as the source of employee accounts and groups
- **OpenID Connect (OIDC)** so internal web applications use **single sign-on (SSO)**

Employees sign in once with their **existing AD username and password**. Keycloak validates credentials against AD (via LDAP). Each business application trusts Keycloak and receives OIDC tokens instead of maintaining separate user databases.

Yes - Active Directory is fully in scope. Keycloak does not replace AD. AD remains where IT creates and disables users. Keycloak **federates** AD: it reads users and groups and performs login validation over the LDAP protocol.

2. Client use case

2.1 Problem today

Issue | Impact

Each applica

2.2 Target outcome

Goal | Solution

One employ

2.3 Typical project scope

1. Install Keycloak (VM or Docker) with PostgreSQL database
2. Configure **LDAP user federation** to Active Directory
3. Map AD groups to Keycloak roles (optional per app)
4. Register each application as an **OIDC client**
5. TLS (HTTPS), backup/restore, admin runbooks
6. Optional ongoing support and upgrades

3. Components explained

3.1 Active Directory (AD)

Active Directory is Microsoft's directory service for Windows networks. It stores:

- **Users** - login names, passwords (hashed), email, department
- **Groups** - e.g. HR, Finance, IT-Admins
- **Organizational Units (OUs)** - structure like folders
- **Domain Controllers (DC)** - servers that authenticate users

Example domain: `company.local`

Example user: `ahmed` or `ahmed@company.local` (UPN)

AD is the **system of record** for people. The IAM project does not move users out of AD.

3.2 LDAP - how Keycloak talks to AD

LDAP (Lightweight Directory Access Protocol) is the standard protocol directories use.

- Active Directory **speaks LDAP**
- Keycloak **User Federation** uses LDAP to:
- Search for users
- Validate passwords (bind)
- Read group membership

Production connection examples:

- `ldap://dc01.company.local:389`
- `ldaps://dc01.company.local:636` (TLS recommended)

A **service account** in AD (e.g. `svc-keycloak`) is used for searches - not each employee's admin rights.

3.3 Keycloak (on-prem IAM)

Keycloak is an open-source identity server. In this project it provides:

Function | Description

Login UI | E

Keycloak stores **configuration** and **sessions** in PostgreSQL. User passwords for federated users remain in AD (Keycloak can validate via LDAP bind).

3.4 OIDC (OpenID Connect)

OIDC is the modern standard for application login, built on OAuth 2.0.

Artifact | Purpose

Authorizatio

OIDC is used for **web applications** (React, Angular, Spring Boot, .NET, etc.). The demo uses the **Authorization Code flow with PKCE** (recommended for browser apps).

4. Active Directory integration flow

4.1 What IT configures in AD

1. Create **service account** for Keycloak LDAP bind (read users/groups)
2. Allow network path: Keycloak server -> Domain Controller (389 or 636)
3. Confirm attribute used for login (`sAMAccountName` or `userPrincipalName`)
4. Identify groups that should map to application roles

4.2 What is configured in Keycloak

1. **User Federation** -> Add LDAP provider
2. Connection URL, bind DN, bind password
3. Users DN (e.g. `OU=Users,DC=company,DC=local`)
4. Username LDAP attribute (often `sAMAccountName`)
5. **Sync** or **import** users from AD
6. **Group mapper** - AD group -> Keycloak group/role

4.3 Login validation sequence (AD + Keycloak)

```
Employee          Application        Keycloak             Active Directory
                                                                          |
```

4.4 User lifecycle

Event | AD action | Effect in Keycloak / apps

New hire |

5. OIDC flow (detailed)

5.1 Roles

Role | Example in demo

Resource O

5.2 Authorization Code + PKCE (step by step)

PKCE protects public browser apps that cannot store a client secret.

Step | Actor | Action

1 | User |

5.3 ID token claims (example)

Applications read JWT claims such as:

- `sub` - stable user identifier
- `preferred_username` - login name
- `email` - mail address
- `name` - display name
- Custom claims - roles, department (via Protocol Mapper)

5.4 OIDC endpoints (Keycloak)

Endpoint | Path (realm company)

Authorizatio

Apps can use **discovery document** to auto-configure URLs.

6. Single sign-on (SSO) across applications

6.1 First application login

User logs in at Keycloak when opening **HR Portal**. Keycloak creates an **SSO session** (cookie on Keycloak domain).

6.2 Second application without re-login

User opens **Finance Portal** (different OIDC client, same realm):

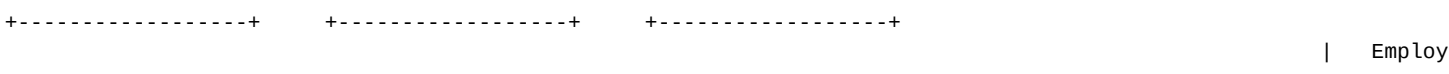
1. Finance app redirects to Keycloak
2. Keycloak sees valid SSO session
3. Keycloak issues new authorization code **without password prompt**
4. Finance app exchanges code for tokens
5. User is signed in - **SSO demonstrated**

6.3 Logout

App-only logout - ends local app session; SSO session may remain.

SSO logout - Keycloak end-session endpoint logs user out of IdP and can propagate to clients (configure front-channel / back-channel logout per app).

7. End-to-end architecture



7.1 Network zones (production)

Zone | Components

Corporate L

Typical hostname: `auth.company.com` -> reverse proxy -> Keycloak.

8. Portfolio demo vs client production

Aspect | Demo (keycloak-ad-sso-demo) | Client production

Directory |

The integration pattern is identical. Only connection details and hardening differ.

9. Delivery checklist

| Deliverable

1 | Keycloa

10. Glossary

Term | Definition

AD | Active

11. References and demo

Resource	Location
Runnable de	

This document is a portfolio technical guide. Customer names and environments are illustrative.